[Nombre de la institución]

[Carrera / Materia]

Canciones con Historia

Aplicación móvil individual

Documento de planeación, arquitectura y configuración del entorno

Caso de estudio · Metodología ágil Kanban · Flutter + Firebase

Alumno:

Matrícula:

Docente:

Grupo:

Fecha: Octubre de 2026

Índice

1. Caso de estudio
2. Plan del proceso de desarrollo móvil (Kanban)
3. Justificación de la arquitectura
4. Diagrama de la arquitectura
5. Propuesta de patrones de diseño
6. Justificación de los frameworks de desarrollo móvil
7. Esquema de pruebas
8. Estrategia de versionamiento
9. Configuración del entorno
    a. Lista de herramientas utilizadas
    b. Parámetros de configuración
10. Conclusiones

1. Caso de estudio

1.1 Contexto

Muchas canciones populares nacieron como respuesta a sucesos trágicos: guerras, atentados, represión o injusticias sociales. Con frecuencia su mensaje se expresa con metáforas o en doble sentido, por lo que quien las escucha no siempre conoce la historia que hay detrás. Un ejemplo es *"Burka azul"*, que hace referencia a la situación de Afganistán mediante dobles sentidos, o *"Zombie"* de The Cranberries, escrita tras el atentado de Warrington de 1993.

1.2 Problema

La información sobre el contexto histórico de estas canciones está dispersa en artículos, entrevistas y videos. No existe un espacio que reúna en un mismo lugar la canción, su letra, el suceso que representa, la interpretación de lo que dice "entre líneas" y el lugar donde se originó.

1.3 Solución propuesta

Canciones con Historia es una aplicación móvil para Android, desarrollada con Flutter y Firebase, que funciona como catálogo curado de canciones con contexto histórico. El acceso se realiza con la cuenta de Google y existen dos roles:

Rol	Funcionalidades
Usuario	Iniciar sesión con Google · consultar y buscar canciones (por título, banda o suceso) · escuchar la canción (reproductor integrado o enlace a YouTube/Spotify) · leer la letra, el contexto del suceso y el significado oculto · agregar y quitar canciones de favoritos · ver en un mapa el lugar donde se creó la canción.
Administrador	Todo lo anterior, más: agregar canciones mediante un formulario (nombre, banda, año, género, imagen, suceso, contexto, doble sentido, letra, enlaces de audio y ubicación en el mapa) · editar canciones · eliminar canciones.

1.4 Alcance y restricciones

- Plataforma objetivo: Android (el código Flutter permite compilar también para iOS y Web en el futuro).
- Proyecto individual con presupuesto cero: se usan únicamente servicios gratuitos (plan Spark de Firebase, mapas de OpenStreetMap).
- Los administradores se identifican por una lista de correos autorizados, validada tanto en la app como en las reglas de seguridad del servidor.
- Por derechos de autor, la app no aloja archivos de audio comerciales: reproduce enlaces directos o redirige a plataformas oficiales.

2. Plan del proceso de desarrollo móvil (Kanban)

2.1 Justificación de la metodología

Se seleccionó **Kanban** porque el proyecto es individual: no hay un equipo que requiera ceremonias de Scrum (Sprint Planning, Daily, roles de Scrum Master y Product Owner). Kanban permite un flujo continuo de trabajo, visualizar el avance de cada funcionalidad, limitar el trabajo en curso para terminar tareas antes de empezar otras y adaptarse a cambios de prioridad (por ejemplo, la configuración de Firebase surgió como tarea bloqueante a mitad del desarrollo).

2.2 Principios aplicados

1. **Visualizar el flujo:** tablero con columnas que representan cada etapa del trabajo.
2. **Limitar el trabajo en curso (WIP):** máximo 2 tarjetas en "En desarrollo" y 2 en "Pruebas".
3. **Gestionar el flujo:** medir cuánto tarda cada tarjeta en terminarse y detectar cuellos de botella.
4. **Políticas explícitas:** cada columna tiene una "definición de terminado".
5. **Mejora continua:** revisión semanal del tablero para ajustar prioridades.

2.3 Tablero Kanban



Figura 1. Tablero Kanban del proyecto (estado representativo durante la fase de pruebas).

Políticas de cada columna (definición de terminado)

Columna	Criterio para mover la tarjeta a la siguiente columna
Backlog	Idea o requerimiento registrado con una descripción breve.

Por hacer	Tarjeta priorizada, con criterios de aceptación definidos y sin dependencias bloqueantes.
En desarrollo	Código escrito, `flutter analyze` sin errores y cambios guardados en una rama de Git.
Pruebas	Pruebas unitarias en verde y caso de prueba manual ejecutado en el teléfono físico.
Terminado	Rama integrada a `develop`, funcionalidad verificada en el dispositivo y documentada.

2.4 Backlog del producto (tarjetas)

ID	Tarjeta	Criterio de aceptación	Prioridad
K-01	Configurar proyecto Flutter	La app compila e instala en el teléfono físico Android.	Alta
K-02	Modelo de datos `Song`	Se serializa y deserializa sin pérdida de datos (prueba unitaria).	Alta
K-03	Configurar Firebase	Proyecto creado, `google-services.json` agregado, huella SHA-1 registrada.	Alta
K-04	Inicio de sesión con Google	El usuario entra con su cuenta; los correos autorizados obtienen rol de administrador.	Alta
K-05	Listado y búsqueda	Se listan las canciones en tiempo real y se filtran por título, banda o suceso.	Alta
K-06	Detalle de canción	Muestra imagen, suceso, contexto, doble sentido, letra y ubicación.	Alta
K-07	Reproductor de audio	Reproduce/pausa un enlace mp3 y abre enlaces de YouTube/Spotify.	Media
K-08	Favoritos	El usuario marca y desmarca favoritos; persisten al cerrar sesión.	Alta
K-09	Formulario de administrador	Valida campos obligatorios, permite imagen (galería/cámara) y ubicación en mapa; agrega, edita y elimina.	Alta
K-10	Mapa de canciones	Todas las canciones con ubicación aparecen como marcadores.	Media
K-11	Reglas de seguridad de Firestore	Solo administradores escriben canciones; cada usuario solo accede a sus favoritos.	Alta
K-12	Pruebas de widgets e integración	Flujos principales cubiertos con pruebas automatizadas.	Media
K-13	Build de release y firma	APK/AAB firmado con clave propia y versión incrementada.	Media

2.5 Cronograma estimado

Kanban no trabaja con iteraciones fijas; sin embargo, para fines de planeación se agrupan las tarjetas en fases semanales y se realiza una revisión del tablero al final de cada semana.

Semana	Fase	Tarjetas
1	Análisis y configuración	Caso de estudio, requerimientos, K-01, K-02
2	Autenticación y datos	K-03, K-04, K-11
3	Funcionalidad del usuario	K-05, K-06, K-07, K-08
4	Funcionalidad del administrador	K-09, K-10
5	Pruebas y estabilización	K-12, corrección de defectos, carga de contenido
6	Entrega	K-13, documentación final y presentación

2.6 Métricas de seguimiento

- **Lead time:** tiempo desde que la tarjeta entra a “Por hacer” hasta “Terminado”.
- **Cycle time:** tiempo que la tarjeta permanece en “En desarrollo” y “Pruebas”.
- **Throughput:** número de tarjetas terminadas por semana.
- **Tarjetas bloqueadas:** se marcan en rojo con la causa (ej. “falta habilitar Google en Firebase”).

Herramienta propuesta para el tablero: **GitHub Projects** (vista de tablero), ya que se integra con el repositorio y permite vincular cada tarjeta con su rama y sus commits. Alternativa: Trello.

3. Justificación de la arquitectura

3.1 Arquitectura seleccionada

Se adoptó una **arquitectura cliente – Backend as a Service (BaaS)**, y dentro de la app una **arquitectura en capas** inspirada en MVVM:

Capa	Ubicación en el código	Responsabilidad
Presentación (Vista)	`lib/screens/`, `lib/widgets/`	Pantallas y componentes visuales. No acceden directamente a Firebase.
Estado (ViewModel)	`lib/state/app_state.dart`	Mantiene el usuario actual, la lista de canciones y los favoritos; notifica a la vista cuando cambian.
Datos (Repositorio)	`lib/services/backend.dart`	Interfaz `Backend` con dos implementaciones: `FirebaseBackend` y `LocalBackend` (modo demo).
Modelo	`lib/models/song.dart`	Entidades `Song` y `AppUser`, y su conversión a/desde mapas.
Servicios externos	Nube	Firebase Authentication, Cloud Firestore, teselas de OpenStreetMap, servidores de audio y YouTube/Spotify.

3.2 Razones de la elección

- **Sin servidor propio:** Firebase resuelve autenticación, base de datos y seguridad. Para un proyecto individual elimina el costo de programar, hospedar y mantener una API.
- **Tiempo real:** Firestore envía los cambios al instante. Cuando un administrador agrega una canción, aparece en los teléfonos de todos los usuarios sin recargar.
- **Seguridad del lado del servidor:** los permisos no dependen solo de ocultar botones en la app; las *reglas de Firestore* validan en el servidor que solo los correos autorizados puedan escribir canciones.
- **Separación de responsabilidades:** las pantallas no saben si los datos vienen de Firebase o del almacenamiento local. Esto facilita las pruebas y permite cambiar de proveedor modificando una sola capa.
- **Modo demo (tolerancia a fallos de configuración):** si Firebase no está configurado, la app arranca con `LocalBackend` y guarda datos en el teléfono. Esto permitió desarrollar y probar la interfaz antes de tener el proyecto de Firebase.
- **Costo cero:** las imágenes se comprimen (máx. 900 px, calidad 70 %) y se guardan en Firestore como Base64, evitando Firebase Storage, que requiere el plan de pago. Los mapas usan OpenStreetMap, que no requiere clave de API.

3.3 Modelo de datos (Cloud Firestore)

```
songs/{songId}
  title, artist, year, genre        → datos de la canción
  event, history, meaning, lyrics   → suceso, contexto, doble sentido, letra
  imageBase64                       → imagen comprimida
  audioUrl, externalUrl             → mp3 directo / YouTube o Spotify
  placeName, latitude, longitude    → ubicación donde se creó
  createdAt                         → fecha de alta (orden del listado)

users/{uid}
  name, email, lastLogin
  favorites: [songId, songId, ...]
```

4. Diagrama de la arquitectura

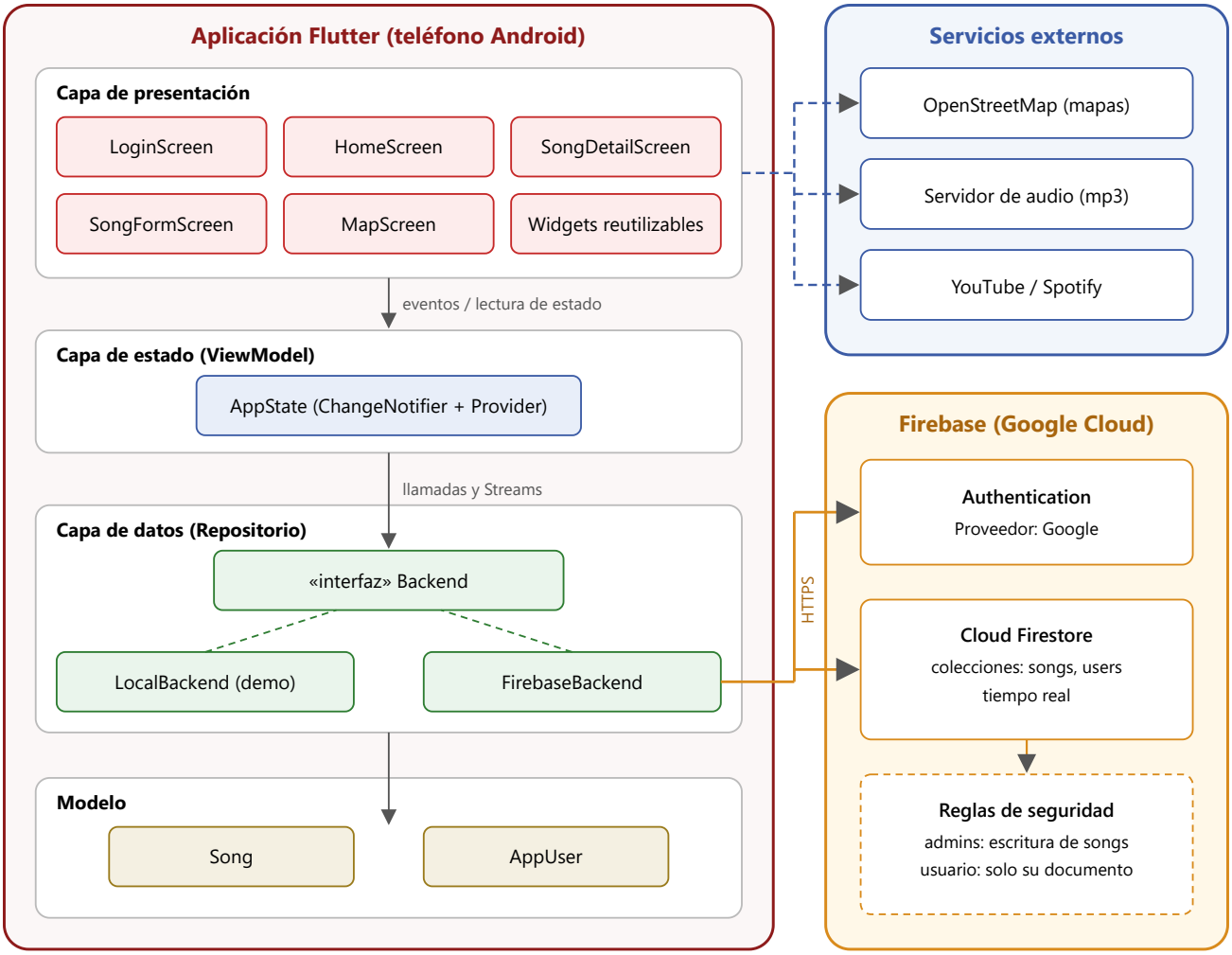


Figura 2. Arquitectura en capas de la aplicación y su relación con Firebase y los servicios externos.

4.1 Flujo de una operación (agregar canción)

1. El administrador llena el formulario en `SongFormScreen`; los validadores verifican los campos obligatorios y el formato de los enlaces.
2. La pantalla construye un objeto `Song` y llama a `backend.saveSong()`.
3. `FirebaseBackend` convierte el objeto con `toMap()` y lo envía a la colección `songs` de Firestore.
4. Las reglas de seguridad verifican que el correo del usuario autenticado esté en la lista de administradores.
5. Firestore notifica el cambio por el `Stream` de canciones; `AppState` actualiza su lista y llama a `notifyListeners()`.
6. Todas las pantallas que observan el estado se redibujan con la nueva canción, en todos los dispositivos conectados.

5. Propuesta de patrones de diseño

Patrón	Tipo	Dónde se aplica	Justificación
Repository	Arquitectónico	Interfaz `Backend` (`lib/services/backend.dart`)	Aísla el acceso a datos. Las pantallas piden "las canciones" sin saber si vienen de Firestore o del teléfono.
Strategy	Comportamiento	`FirebaseBackend` / `LocalBackend`, elegido en `main.dart`	Permite cambiar el algoritmo de persistencia en tiempo de ejecución: si Firebase no inicializa, se usa la estrategia local (modo demo).
Observer	Comportamiento	`Stream` de Firestore, `ChangeNotifier` y `Consumer` / `context.watch`	Las vistas se suscriben al estado y se actualizan automáticamente cuando cambian los datos, sin recargar manualmente.
MVVM	Arquitectónico	Vistas (`screens`) + ViewModel (`AppState`) + Modelo (`Song`)	Separa la lógica de presentación de la interfaz; las pantallas solo dibujan y delegan acciones.
Inyección de dependencias (Provider)	Estructural	`ChangeNotifierProvider` en `MiApp`	El backend se crea una vez y se inyecta en `AppState`; las pantallas lo obtienen del árbol de widgets. Facilita sustituirlo por un doble de prueba.
Factory Method	Creacional	`Song.fromMap()`, `_createBackend()`	Centraliza la creación de objetos a partir de documentos de Firestore o de JSON, con valores por defecto seguros.
Singleton	Creacional	`FirebaseAuth.instance`, `FirebaseFirestore.instance`	Una sola conexión compartida con Firebase en toda la app.
Composite	Estructural	Árbol de widgets de Flutter	Las pantallas se componen de widgets reutilizables (`SongCard`, `SongImage`, `OsmMap`, `AudioPlayerCard`).
Actualización optimista	Patrón de UI	`AppState.toggleFavorite()`	El corazón de favorito cambia al instante y la escritura en el servidor ocurre en segundo plano, mejorando la percepción de velocidad.

5.1 Ejemplo: Repository + Strategy

```
abstract class Backend {                       // Repository (contrato)
  Stream<List<Song>> songs();
  Future<void> saveSong(Song song);
  Future<void> setFavorite(String uid, String songId, bool fav);
  ...
}
class FirebaseBackend implements Backend { ... }   // Estrategia 1: nube
class LocalBackend    implements Backend { ... }   // Estrategia 2: modo demo

Future<Backend> _createBackend() async {         // Selección en tiempo de ejecución
  try { await Firebase.initializeApp(); return FirebaseBackend(); }
  catch (_) { final l = LocalBackend(); await l.init(); return l; }
}
```

6. Justificación de los frameworks de desarrollo móvil

6.1 Comparativa de opciones

Criterio	Flutter (elegido)	React Native	Nativo (Kotlin)
Lenguaje	Dart	JavaScript / TypeScript	Kotlin
Multiplataforma	Android, iOS, Web y escritorio con un solo código	Android e iOS	Solo Android
Rendimiento	Alto: compilado a código nativo (AOT) y motor de render propio	Bueno, con puente o JSI hacia componentes nativos	Máximo
Interfaz	Material 3 incluido; idéntica en todos los dispositivos	Depende de componentes nativos de cada plataforma	Jetpack Compose
Productividad	Hot reload, tipado fuerte, analizador integrado	Fast refresh	Más código por pantalla
Integración con Firebase	Plugins oficiales (FlutterFire)	Librería de terceros (React Native Firebase)	SDK oficial

6.2 Frameworks y librerías seleccionados

Framework / paquete	Versión	Uso y justificación
Flutter SDK	3.44.2	Framework principal de UI. Un solo código base, rendimiento nativo y Material 3 para un diseño moderno con tema oscuro.
Dart	3.12.2	Lenguaje con *null safety*, que evita errores por valores nulos desde la compilación.
firebase_core	^4.15.0	Inicialización de Firebase en la app.
firebase_auth	^6.7.0	Inicio de sesión con Google (`signInWithProvider`) sin administrar contraseñas.
cloud_firestore	^6.10.0	Base de datos NoSQL en tiempo real, con soporte sin conexión (caché local).
provider	^6.1.5	Gestión de estado recomendada por el equipo de Flutter; sencilla para un proyecto de este tamaño.
shared_preferences	^2.5.6	Almacenamiento local clave-valor para el modo demo.
image_picker	^1.2.3	Seleccionar imagen desde galería o cámara, con compresión integrada.

audioplayers	^6.8.1	Reproducción de audio por URL con controles de reproducción, pausa y avance.
flutter_map + latlong2	^8.3.2 / ^0.10.1	Mapas con OpenStreetMap: gratuitos y sin clave de API (a diferencia de Google Maps).
url_launcher	^6.3.3	Abrir enlaces de YouTube o Spotify en su aplicación.
flutter_lints	^6.0.0	Reglas de estilo y buenas prácticas verificadas por `flutter analyze`.
flutter_test	SDK	Pruebas unitarias y de widgets.

6.3 Backend: ¿por qué Firebase?

- Inicio de sesión con Google integrado: requisito principal del caso de estudio.
- Plan gratuito (Spark) suficiente para el volumen esperado del proyecto.
- Reglas de seguridad declarativas que permiten implementar los roles de administrador y usuario sin un servidor propio.
- Alternativas descartadas: una API propia (Node.js/PHP + MySQL) exigiría hospedaje y mantenimiento; Supabase es viable, pero su integración con el inicio de sesión de Google en Android requiere más configuración.

7. Esquema de pruebas

7.1 Niveles de prueba



Figura 3. Pirámide de pruebas: muchas pruebas rápidas en la base y pocas pruebas costosas en la cima.

Nivel	Herramienta	Qué se prueba
Análisis estático	`flutter analyze` + flutter_lints	Errores de tipos, código no usado y malas prácticas. Se ejecuta antes de cada commit.
Unitarias	`flutter_test`	Serialización de `Song` (`toMap` / `fromMap`), validadores del formulario (año, URL, campos obligatorios), función `isAdminEmail`, lógica de favoritos de `AppState` con `LocalBackend`.
Widgets	`flutter_test` (`testWidgets`)	Que la pantalla de login muestre los botones correctos según el modo; que el formulario muestre errores si faltan campos; que el botón "Agregar" solo aparezca para administradores.
Integración	`integration_test` en dispositivo	Flujo completo: iniciar sesión (modo demo) → agregar canción → verla en el listado → marcar favorito → verla en Favoritos.
Reglas de seguridad	Firebase Emulator Suite	Que un usuario no administrador reciba *permission-denied* al escribir en `songs`, y que nadie lea los favoritos de otro usuario.
Manuales / aceptación	Teléfono físico Android 16	Casos de prueba de la sección 7.2, con capturas como evidencia.

7.2 Casos de prueba manuales

ID	Caso	Pasos	Resultado esperado
CP-01	Inicio de sesión como usuario	Tocar "Continuar con Google" y elegir una cuenta no administradora.	Entra al listado sin etiqueta ADMIN ni botón "Agregar".

CP-02	Inicio de sesión como administrador	Iniciar sesión con un correo de la lista de administradores.	Aparecen la etiqueta ADMIN y el botón "Agregar".
CP-03	Agregar canción	Llenar el formulario con imagen y ubicación y tocar "Agregar canción".	Mensaje "Canción agregada"; aparece en el listado y en Firestore.
CP-04	Validación del formulario	Dejar vacío el nombre o escribir un enlace sin `https://`.	Los campos se marcan en rojo y no se guarda.
CP-05	Editar canción	En el detalle, tocar el lápiz, cambiar el año y guardar.	El cambio se refleja de inmediato en el detalle y en el listado.
CP-06	Eliminar canción	Tocar el bote de basura y confirmar.	La canción desaparece del listado y del mapa.
CP-07	Favoritos	Tocar el corazón, cerrar sesión y volver a entrar.	La canción sigue en la pestaña Favoritos.
CP-08	Reproducir audio	Abrir una canción con enlace mp3 y tocar reproducir.	Se escucha el audio y avanza la barra de progreso.
CP-09	Enlace externo	Tocar "Escuchar en YouTube / Spotify".	Se abre la aplicación correspondiente.
CP-10	Mapa	Abrir la pestaña Mapa y tocar un marcador.	Se abre el detalle de esa canción.
CP-11	Búsqueda	Escribir parte del nombre de un suceso.	Solo se muestran las canciones que coinciden.
CP-12	Seguridad	Intentar escribir en `songs` con una cuenta no autorizada.	Firestore rechaza con *permission-denied*.

7.3 Registro de defectos encontrados

Defecto	Causa	Solución
La compilación de Android se detenía (JVM sin memoria).	Gradle reservaba 8 GB de RAM.	Se redujo a 3 GB en `gradle.properties`.
Error `CONFIGURATION_NOT_FOUND` al iniciar sesión.	El proveedor de Google no estaba habilitado en Firebase Authentication.	Se habilitó Google y se volvió a descargar `google-services.json`.
`PERMISSION_DENIED` al guardar una canción.	Las reglas de Firestore no estaban publicadas.	Se publicaron las reglas con la lista de administradores.

7.4 Ejemplo de prueba unitaria implementada

```
test('Song se serializa y deserializa', () {
  final s = Song(id: 'x', title: 'Burka azul', artist: 'Banda', year: 2001,
                 latitude: 34.5, longitude: 69.2, createdAt: DateTime(2024));
  final back = Song.fromMap('x', s.toMap());
  expect(back.title, 'Burka azul');
  expect(back.year, 2001);
  expect(back.hasLocation, isTrue);
});
```

Comandos de ejecución: `flutter analyze` · `flutter test` · `flutter test integration_test`.

8. Estrategia de versionamiento

8.1 Control de versiones del código

Se utiliza **Git** (2.52) con repositorio remoto en **GitHub**, siguiendo un modelo **Git Flow simplificado**, adecuado para un desarrollador individual:

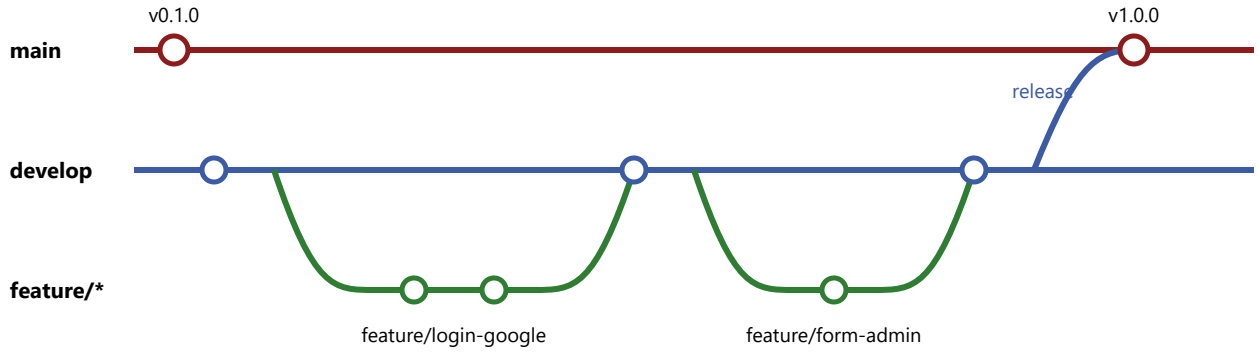


Figura 4. Flujo de ramas: las funcionalidades se desarrollan en ramas feature y se integran a develop; las versiones estables se publican en main con una etiqueta.

Rama	Propósito
`main`	Código estable y entregable. Cada versión publicada se marca con una etiqueta (`v1.0.0`).
`develop`	Integración de funcionalidades terminadas (columna "Terminado" del tablero Kanban).
`feature/<nombre>`	Una rama por tarjeta Kanban, por ejemplo `feature/K-08-favoritos` .
`hotfix/<nombre>`	Correcciones urgentes sobre `main` , que después se integran también a `develop` .

Convención de mensajes de commit (Conventional Commits)

```
feat: agregar formulario de canciones para administrador
fix: corregir permiso al guardar canción en Firestore
docs: documentar configuración de Firebase
test: agregar prueba de serialización de Song
chore: reducir memoria de Gradle a 3 GB
```

8.2 Versionamiento de la aplicación (SemVer)

La versión se define en `pubspec.yaml` con el formato `MAYOR.MENOR.PARCHE+COMPILACIÓN` . Actualmente: `version: 1.0.0+1` .

Componente	Cuándo se incrementa	Ejemplo

MAYOR	Cambios incompatibles, por ejemplo una nueva estructura de la base de datos que requiere migración.	1.4.2 → 2.0.0
MENOR	Nueva funcionalidad compatible, por ejemplo agregar comentarios o notificaciones.	1.0.0 → 1.1.0
PARCHE	Corrección de errores sin cambiar funcionalidades.	1.1.0 → 1.1.1
+COMPILACIÓN	Se incrementa en cada APK/AAB generado. En Android corresponde a `versionCode` y debe ser siempre mayor que el anterior para publicar en Google Play.	+1 → +2

Proceso de liberación

1. Actualizar `version` en `pubspec.yaml` y registrar los cambios en `CHANGELOG.md`.
2. Ejecutar `flutter analyze` y `flutter test` (todo en verde).
3. Integrar `develop` en `main` y crear la etiqueta: `git tag v1.0.0`.
4. Generar el instalable: `flutter build apk --release` (o `appbundle` para Google Play).

9. Configuración del entorno

9.1 Lista de herramientas utilizadas

Herramienta	Versión	Uso
Windows 11 Home	25H2 (10.0.26200)	Sistema operativo de desarrollo.
Flutter SDK (canal stable)	3.44.2	Framework de desarrollo; instalado en `C:\flutter`.
Dart SDK	3.12.2	Lenguaje de programación (incluido en Flutter).
Flutter DevTools	2.57.0	Depuración, inspector de widgets y rendimiento.
Visual Studio Code	Última estable	Editor de código con extensiones Flutter y Dart.
Android SDK	36.1.0	Plataforma `android-36.1` y build-tools 36.1.0.
Android Platform Tools (adb)	Incluido en el SDK	Instalación y registros (`logcat`) en el teléfono.
Android Emulator	37.1.11	Emulación opcional (se usó principalmente un dispositivo físico).
Java JDK (Eclipse Temurin)	21.0.11	Requerido por Gradle para compilar Android.
Gradle	9.1.0	Sistema de compilación de Android.
Android Gradle Plugin	9.0.1	Plugin de compilación de Android.
Kotlin Gradle Plugin	2.3.20	Compilación del código Kotlin de los plugins.
Google Services Plugin	4.4.4	Lee `google-services.json` para conectar con Firebase.
Firebase Console	Web	Proyecto `canciones-con-historia`: Authentication, Firestore y reglas.
Git	2.52.0	Control de versiones.
GitHub / GitHub Projects	Web	Repositorio remoto y tablero Kanban.
Node.js	26.4.0	Opcional: Firebase CLI y emulador para probar reglas de seguridad.
Google Chrome / Edge	154	Ejecución web de pruebas y consola de Firebase.
Dispositivo de pruebas	Xiaomi 2311DRK48G, Android 16 (API 36)	Pruebas en teléfono físico por USB.

9.2 Parámetros de configuración

a) Proyecto Flutter (`pubspec.yaml`)

```
name: mi_app
version: 1.0.0+1
environment:
  sdk: ^3.12.2
dependencies:
  firebase_core: ^4.15.0      cloud_firestore: ^6.10.0
  firebase_auth: ^6.7.0       provider: ^6.1.5+1
  shared_preferences: ^2.5.6  image_picker: ^1.2.3
  audioplayers: ^6.8.1        flutter_map: ^8.3.2
  latlong2: ^0.10.1           url_launcher: ^6.3.3
```

b) Android (`android/app/build.gradle.kts`)

Parámetro	Valor
`applicationId` / `namespace`	`com.example.mi_app` (debe coincidir con el registrado en Firebase)
`compileSdk`, `targetSdk`, `minSdk`	Valores por defecto de Flutter 3.44 (`flutter.compileSdkVersion`, etc.)
Compatibilidad Java / Kotlin	`JavaVersion.VERSION_17` / `JvmTarget.JVM_17`
Plugin de Google Services	Se aplica solo si existe `google-services.json`; si no, la app inicia en modo demo.
Firma de release	Temporalmente con la clave de depuración; para Google Play se creará un *keystore* propio.

```
// Firebase: solo se activa cuando existe android/app/google-services.json
if (file("google-services.json").exists()) {
    apply(plugin = "com.google.gms.google-services")
}
```

c) Gradle (`android/gradle.properties`)

```
org.gradle.jvmargs=-Xmx3G -XX:MaxMetaspaceSize=1G -XX:ReservedCodeCacheSize=256m
kotlin.daemon.jvmargs=-Xmx1536m
android.useAndroidX=true
```

La memoria se limitó a 3 GB porque con la configuración por defecto (8 GB) la máquina virtual de Java se quedaba sin memoria durante la compilación.

d) Manifiesto (`AndroidManifest.xml`)

Parámetro	Valor / motivo

`uses-permission INTERNET`	Firebase, mapas y audio por URL.
`android:label`	"Canciones con Historia".
`queries → VIEW https`	Necesario en Android 11+ para que `url_launcher` abra YouTube/Spotify.

e) Firebase

Parámetro	Valor
Proyecto	`canciones-con-historia` (plan Spark, gratuito)
App Android registrada	Paquete `com.example.mi_app`
Huella SHA-1 (depuración)	`55:29:9E:73:C9:89:09:F5:CF:AC:37:1F:39:B4:A2:CE:11:69:82:D3`
Archivo de configuración	`android/app/google-services.json` (no se sube a repositorios públicos)
Authentication	Proveedor **Google** habilitado
Cloud Firestore	Edición Standard, base de datos `(default)`, modo producción
Administradores	Lista de 3 correos en `lib/config.dart` y en `firestore.rules` (deben coincidir)

f) Reglas de seguridad (`firestore.rules`)

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    function isAdmin() {
      return request.auth != null &&
        request.auth.token.email in [ /* correos de administradores */ ];
    }
    match /songs/{songId} {
      allow read:  if request.auth != null;   // cualquier usuario con sesión
      allow write: if isAdmin();              // solo administradores
    }
    match /users/{uid} {                       // favoritos de cada usuario
      allow read, write: if request.auth != null && request.auth.uid == uid;
    }
  }
}
```

g) Comandos principales

Comando	Función
`flutter pub get`	Descargar dependencias.
`flutter devices`	Listar dispositivos conectados.

`flutter run -d <id>`	Ejecutar la app en un dispositivo.
`flutter analyze`	Análisis estático.
`flutter test`	Ejecutar pruebas.
`flutter build apk --release`	Generar el instalable APK.
`keytool -list -v -keystore %USERPROFILE%\.android\debug.keystore`	Obtener la huella SHA-1 para Firebase (contraseña `android`).

h) Estructura del proyecto

```
mi_app/
├── android/app/google-services.json   ← configuración de Firebase
├── lib/
│   ├── main.dart                      ← arranque y selección de backend
│   ├── config.dart                    ← correos de administradores
│   ├── theme.dart                     ← tema oscuro (Material 3)
│   ├── models/song.dart               ← Song, AppUser
│   ├── services/backend.dart          ← Backend, FirebaseBackend, LocalBackend
│   ├── services/seed_songs.dart       ← datos de ejemplo (modo demo)
│   ├── state/app_state.dart           ← estado global (ChangeNotifier)
│   ├── screens/                       ← login, home, detalle, formulario, mapa
│   └── widgets/                       ← tarjetas, mapa, reproductor
├── test/widget_test.dart              ← pruebas
├── firestore.rules                    ← reglas de seguridad
└── pubspec.yaml                       ← dependencias y versión
```

10. Conclusiones

El proyecto **Canciones con Historia** demuestra que una aplicación móvil completa —con autenticación, roles, base de datos en tiempo real, multimedia y mapas— puede desarrollarse de forma individual y sin costo gracias a la combinación de Flutter y Firebase.

Kanban resultó adecuado para un desarrollador individual: permitió visualizar el avance, atender de inmediato las tareas bloqueantes (configuración de Firebase, reglas de seguridad, memoria de compilación) y mantener un flujo continuo sin la sobrecarga de ceremonias de otras metodologías.

La arquitectura en capas y los patrones Repository, Strategy y Observer dieron como resultado un código desacoplado: la interfaz no depende de Firebase, lo que permitió desarrollar primero en modo demo y facilita tanto las pruebas como futuras extensiones.

Como trabajo futuro se plantea completar las pruebas de widgets e integración, firmar la aplicación con una clave propia para publicarla en Google Play, y agregar funciones como comentarios de usuarios, notificaciones de nuevas canciones y una versión para iOS.